

MCQ :

20 Questions 1 HR time

15 C + 5 Java + 5 aptitude

- 1) Check whether a number is odd without using modulo operation.
 - a) $x\%2$?odd : even
 - b) $x\&1$
 - c) $\sim x$
 - d) $x|1$
- 2) Check if two numbers have opposite signs or not using xor operator.
 - a) $(x^y) < 1$
 - b) $(x^y) > 0$
 - c) $(x^y) < 0$
 - d) $(x^y) > 1$
- 3) Add 1 to an integer without using the addition operator.
 - a) $\sim x$
 - b) $\sim \sim x$
 - c) $!x$
 - d) $!(x^1)$
- 4) Add 1 to an integer without using the addition operator.
 - a) $\sim x$
 - b) $\sim \sim x$
 - c) $!x$
 - d) $!(x^1)$
- 5) How -3 is converted to binary?
 - a) 1000000000.....11
 - b) 111111111111.....01
 - c) Both a & b
- 6) Swap two numbers using bit operations
($a = a \text{ op } b$, $b = a \text{ op } b$, $a = a \text{ op } b$) choose op
 - a) $\wedge, \wedge, \wedge$
 - b) $\&, |, \&$
 - c) $+, -, -$
 - d) Both a & c
- 7) Is the second bit set bit or not
 $N = 13$, $i = 2$
 - a) Yes
 - b) No
- 8) Output?
`#include <stdio.h>`

`int main() {`
`int x = -8;`

```
printf("%d", x >> 2);  
return 0;  
}
```

a) -2
b) 2
c) 1073741822
d) Error : cannot right shift negative numbers

9) #include <stdio.h>

```
int main() {  
    printf("%d", 5 << 2 + 1);  
}
```

a) 21
b) 40
c) 20
d) 22

10) #include <stdio.h>

```
int main() {  
    int x = 12;  
    printf("%d", x >> 1 << 2);  
}
```

a) 24
b) 12
c) 6
d) 8

11) power of 2 or not

- a) $n \& (n - 1) == 1$
- b) $n \& (n - 1) == 0$
- c) $n | (n \& n) == 0$
- d) $n | n \& n == 1$

12) #include <stdio.h>

```
int main() {  
    printf("%d", ~-5);  
}
```

a) 4
b) -4
c) 5
d) -5

13) #include <stdio.h>

```
int main() {  
    printf("%d", 5 ^ 7 ^ 5);  
}
```

- a) 5
- b) 7
- c) 0
- d) 2

14) #include <stdio.h>

```
int main() {  
    printf("%d", 5 | 3 & 6);  
}
```

- a) 7
- b) 6
- c) 5
- d) 3

15) #include <stdio.h>

```
int main() {  
    int x = 12;  
    printf("%d", x & -x);  
}
```

- a) 12
- b) 8
- c) 4
- d) 2

16) #include <stdio.h>

```
int main() {  
    printf("%d", 5 && 2 | 1);  
}
```

- a) 0
- b) 1
- c) 2
- d) 3

17) #include <stdio.h>

```
int main() {  
    printf("%d", 5 && 2 | 1);  
}
```

- a) 0
- b) 1
- c) 2
- d) 3

18) #include <stdio.h>

```
int main() {  
    int x = 8;  
  
    printf("%d", x >> 1 + 1 << 1);  
}
```

- a) 8
- b) 4
- c) 2
- d) 10

19) #include <stdio.h>

```
int main()  
{  
    int x = 5;  
  
    printf("%d", x ^ x & x);  
}
```

- a) 0
- b) 1
- c) 5

20) #include <stdio.h>
#include <stdlib.h>

```
int main() {  
    int *p = (int *)malloc(sizeof(int));  
    printf("%d", *p);  
}
```

- a) 0

- b) Garbage value
- c) NULL
- d) No output

21) #include <stdio.h>

#include <stdlib.h>

```
int main() {
    int *p = (int *)calloc(5, sizeof(int));
    printf("%d", p[3]);
}
```

- A) Garbage
- B) 0
- C) No output
- D) NULL

22) int *p = malloc(100);

p = malloc(200);

- a) Memory gets updated
- b) Memory leak
- c) Segmentation fault
- d) Compilation error

23) int *p = NULL;

free(p);

printf("Done");

- a) Segmentation fault
- b) Runtime error
- c) Prints "Done"
- d) Undefined behavior

24) int *p = malloc(sizeof(int));

free(p);

free(p);

printf("Done");

- e) Segmentation fault
- f) Safe to do
- g) Prints "Done"
- h) Undefined behavior

```
25) int *p = malloc(5 * sizeof(int));  
p = realloc(p, 10 * sizeof(int));
```

What does realloc do?

- a) Always moves memory
- b) Keeps the same address
- c) May keep same address or move to a new one
- d) Frees memory

26) Where is memory allocated by malloc() stored?

- a) Stack
- b) Heap
- c) Data segment
- d) Code segment

```
27) int *p = malloc(10 * sizeof(int));
```

```
printf("%zu", sizeof(p));
```

64 bit system

- a) 40
- b) 80
- c) 4
- d) 8

```
28) int *p = malloc(sizeof(int));
```

```
free(p);
```

```
printf("%d", *p);
```

- i) Segmentation fault
- j) Runtime error
- k) No error prints 8
- l) Undefined behavior

```
29) #include <stdio.h>
```

```
#include <stdlib.h>
```

```
int main() {
```

```
    int *p = malloc(sizeof(int));
```

```
    *p = 10;
```

```
    free(p);
```

```
    if (p)
```

```
        printf("Valid");
```

```
    else
```

```
    printf("NULL");  
}
```

- a) NILL
- b) Valid
- c) Segmentation fault

30) free(NULL)

- a) Runtime error
- b) Compile time error
- c) Safe
- d) Segmentation fault

31) #include <stdio.h>

```
int main() {  
    int *p = NULL;  
    *p = 10;  
    printf("%d", *p);  
    return 0;  
}
```

- a) 10
- b) Segmentation fault
- c) Runtime error
- d) Garbage value

```
32) public class Test {  
    public static void main(String[] args) {  
        try {  
            System.out.print("A");  
            int x = 10 / 0;  
            System.out.print("B");  
        } catch (ArithmeticException e) {  
            System.out.print("C");  
        } finally {  
            System.out.print("D");  
        }  
    }  
}
```

- a) ABCD
- b) ABC
- c) ACD
- d) Throws Arithmetic exception

```

33) class Test {
    static int fun() {
        try {
            return 10;
        } finally {
            return 20;
        }
    }

    public static void main(String[] args) {
        System.out.println(fun());
    }
}

```

- a) 10
- b) 20
- c) Runtime error
- d) Compilation error

```

34) try {
    int x = 10 / 0;
}
catch(Exception e) {
    System.out.print("A");
}
catch(ArithmeticException e) {
    System.out.print("B");
}

```

- a) A
- b) B
- c) A then error
- d) Compilation error
- e) Runtime error

```

35) try {
    int x = 10 / 0;
}
catch (ArithmeticException e) {
    System.out.println("Arithmetic");
}
catch (Exception e) {
    System.out.println("Exception");
}

```


- f) A
- g) B
- h) A then error
- i) Compilation error
- j) Runtime error

```
35) public class Test {  
    public static void main(String[] args) {  
        String s = null;  
  
        try {  
            System.out.print(s.length());  
        }  
        catch(RuntimeException e) {  
            System.out.print("Java");  
        }  
    }  
}
```

- a) 0
- b) Java
- c) Null
- d) Error

```
36) public class Test {  
    public static void main(String[] args) {  
  
        try {  
            System.out.print("A");  
            System.exit(0);  
        }  
        finally {  
            System.out.print("B");  
        }  
    }  
}
```

- a) A
- b) B
- c) AB

```
37) public class Test {  
    public static void main(String[] args) {
```

```

    try {
        throw null;
    }
    catch(NullPointerException e) {
        System.out.print("NPE");
    }
}
}

```

- a) Exception
- b) NPE
- c) Compilation error
- d) Runtime error

38) import java.io.*;

```

public class Test {

    public static void main(String[] args) {

        FileReader f = new FileReader("abc.txt");

    }
}

```

- a) Runs successfully
- b) Compilation error
- c) Runtime error
- d) FileNotFoundException

39) try {

```

}
catch(Throwable t) {

}
catch(Exception e) {

}
}

```

- a) Valid
- b) Runtime error
- c) Compilation error
- d) Depends on JVM

```

40) public class Test {

    public static void main(String[] args) {

        try {
            int a[] = new int[2];

            System.out.println(a[5]);

        }
        catch(ArithmeticException e) {
            System.out.print("A");
        }
        catch(ArrayIndexOutOfBoundsException e) {
            System.out.print("B");
        }
    }
}

```

- a) A
- b) B
- c) Runtime error
- d) Compiler error

```

41) public class Test {

    public static void main(String[] args) {

        try {

            try {
                int x = 10 / 0;
            }
            catch(NullPointerException e) {
                System.out.print("A");
            }

        }
        catch(Exception e) {
            System.out.print("B");
        }
    }
}

```

- a) A
- b) B
- c) AB
- d) Compilation error

42) public class Test {

```
    static int fun() {
```

```
        try {
```

```
            return 1;
```

```
        }
```

```
        finally {
```

```
            System.out.print("Finally ");
```

```
        }
```

```
    }
```

```
    public static void main(String[] args) {
```

```
        System.out.print(fun());
```

```
    }
```

```
}
```

- a) 1 Finally
- b) Finally 1
- c) Finally
- d) Compile error

43) Command to display current working directory?

- a) Cd
- b) Pwd
- c) Ls
- d) Who

44) List all files including hidden files?

- a) ls
- b) ls -l
- c) ls -a
- d) ls -r

45) Command to copy a directory and all its content

- a) Cp

- b) cp -r
- c) mv -r
- d) Copy

46) find a word "error" inside log.txt file?

- a) grep error log.txt
- b) find error log.txt
- c) cat error log.txt
- d) search error log.txt

47) what does this command do?

Chmod 755 [file.sh](#)

- a) Gives everyone full permission
- b) Owner: rwx, Group: r-x, Others: r-x
- c) Owner: rw-, Group: rw-, Others: rw-
- d) Removes execute permission

48) count number of lines

- a) wc file.txt
- b) wc -l file.txt
- c) count file.txt
- d) cat -l file.txt

49) command to terminate a process with PID 2456

- a) Stop 2456
- b) Kill 2456
- c) Exit 2456
- d) Terminate 2456

50) What does this command do?

ls > files.txt

- a) Reads from file.txt
- b) Appends output to file.txt
- c) Redirects output to file.txt overwriting it
- d) Deletes file.txt

51) purpose of | operator

- a) Runs commands in parallel
- b) Send the output of one command as input to another
- c) Redirects output to a file
- d) Stops command execution

52) remove a non empty directory

- a) `rm dir`
- b) `rmdir dir`
- c) `rm -r dir`
- d) Delete dir

53) `echo "Hello" > file.txt`

`echo "World" >> file.txt`

`cat file.txt`

- a) Error as there is no file.txt
- b) Hello/n World
- c) World
- d) Hello

Precedence order

Operator	Meaning
~	Bitwise NOT
* / %	Multiplication, Division
+ -	Addition, Subtraction
<< >>	Shift operators
< <= > >=	Relational
== !=	Equality
&	Bitwise AND
^	Bitwise XOR
&&	Logical AND

Topic	Important Points
malloc()	Allocates memory, leaves it uninitialized
calloc()	Allocates and zero-initializes memory
realloc()	Resizes an allocated block; may move it
free()	Releases heap memory
free(NULL)	Safe, no effect
Double free()	Undefined behavior
Dangling pointer	Pointer refers to freed memory
Memory leak	Allocated memory becomes unreachable

`sizeof(pointer)` Size of the pointer (e.g., 8 bytes on 64-bit), not the allocated block

Heap vs Stack Dynamic allocation uses the heap; local variables are on the stack

Command	Purpose
<code>pwd</code>	Print current directory
<code>ls</code>	List files
<code>ls -a</code>	Show hidden files
<code>ls -l</code>	Long listing format
<code>cd</code>	Change directory
<code>cp</code>	Copy files
<code>cp -r</code>	Copy directories recursively
<code>mv</code>	Move or rename files
<code>rm</code>	Remove files
<code>rm -r</code>	Remove directories recursively
<code>mkdir</code>	Create a directory
<code>rmdir</code>	Remove an empty directory
<code>cat</code>	Display file contents
<code>grep</code>	Search text
<code>find</code>	Find files and directories
<code>wc -l</code>	Count lines
<code>head</code>	Show first lines of a file
<code>tail</code>	Show last lines of a file

<code>chmod</code>	Change file permissions
<code>ps</code>	List running processes
<code>kill</code>	Terminate a process
<code>></code>	Redirect output (overwrite)
<code>>></code>	Redirect output (append)